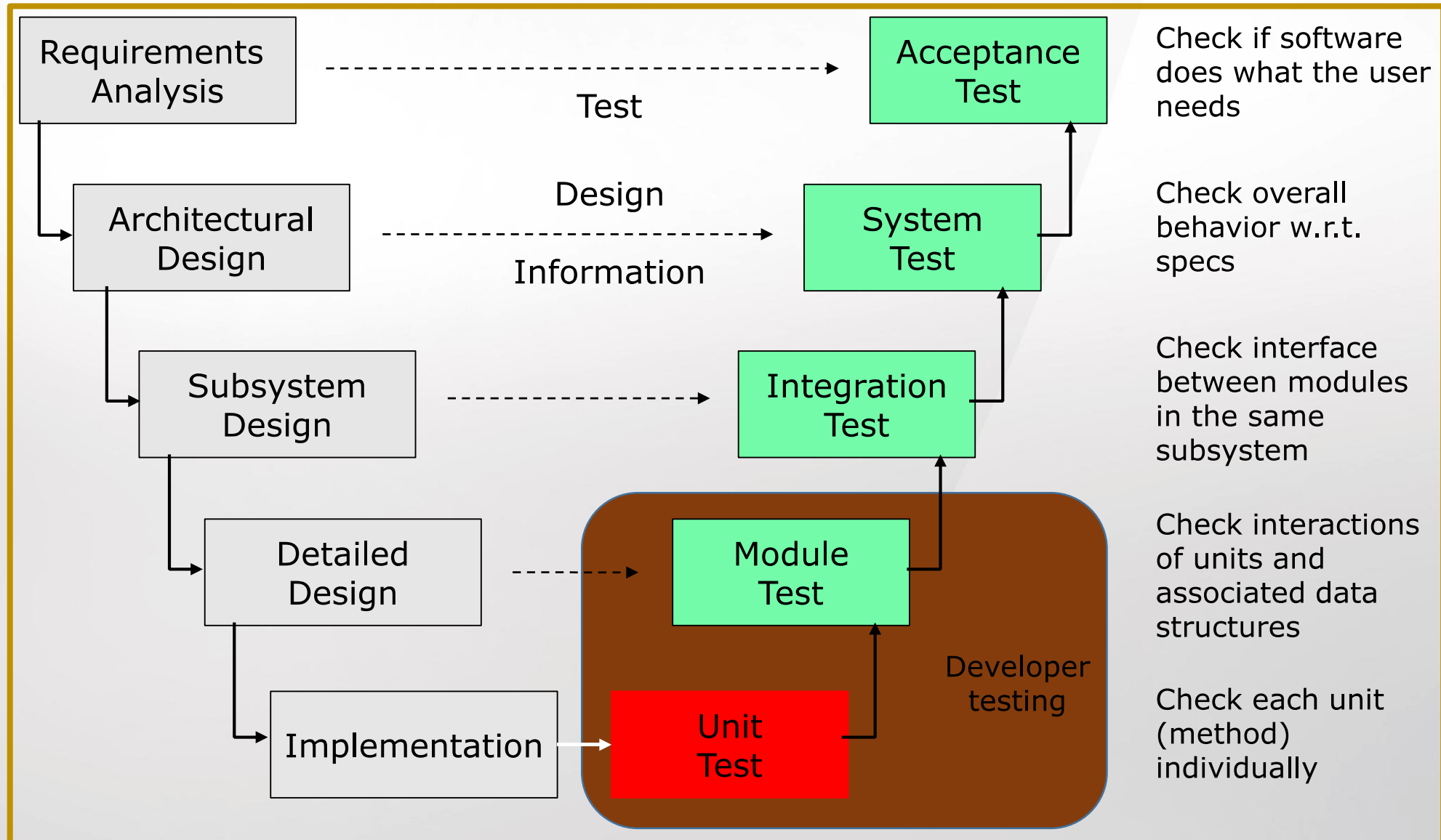


Teste de Caixa Preta

Prof. Iaçanã Ianiski Weber



- ✓ Técnicas:
 - Funcional (Capítulo 2)
 - Estrutural (Capítulo 4)
 - Baseada em Erros (Capítulo 5)
- ✓ A diferença está na fonte utilizada para *definir os requisitos de teste!!*
- ✓ Mas o **objetivo** das técnicas é similar:
 - Conduzir o processo de teste de uma forma mais sistemática
 - Escolher casos de teste que representem o domínio

- ✓ O software é considerado uma **caixa preta**
- ✓ No teste funcional são verificadas as funções do sistema *sem se preocupar com os detalhes de implementação*
- ✓ Entradas e saídas são definidas para verificar conformidade com objetivos ou **especificação**
- ✓ O software é avaliado do ponto de vista do usuário
- ✓ O teste funcional envolve *dois passos principais*:
 1. Identificar as funções que o software deve realizar
 2. Criar casos de teste capazes de checar se essas funções estão sendo realizadas pelo software

1. Particionamento de Equivalência
 2. Análise do Valor Limite
 3. Sistemático
 4. Grafo Causa-Efeito
 5. Error-Guessing
- ✓ A qualidade de tais critérios depende fortemente da existência de uma **boa especificação** de requisitos
 - ✓ Especificações ausentes ou mesmo incompletas tornarão difícil a aplicação dos critérios funcionais

A ideia é dividir o domínio de entrada em classes (partições) onde todos os valores dentro de uma classe devem ter o mesmo comportamento. Se um valor da classe funciona, assume-se que todos os outros também funcionam.

Como funciona:

1. *Identifique as entradas do sistema*
2. *Divida cada entrada em classes válidas e inválidas*
3. *Escolha um representante de cada classe como caso de teste*

Exemplo: campo "idade" aceita 18–65.

Bastam 3 casos de teste (um de cada classe, ex: 10, 30, 70) em vez de testar todos os valores possíveis.

Vantagem: reduz drasticamente o número de testes sem perder cobertura significativa.

CLASSE	FAIXA	TIPO
C1	<18	Inválida
C2	18~65	Válida
C3	>65	Inválida

Complementa o particionamento. A observação é que bugs se concentram nas fronteiras entre as classes, não no meio delas.

Como funciona:

- *Teste os valores exatamente nos limites e imediatamente ao redor deles.*

Exemplo: campo "idade" aceita 18–65.

São 6 casos de teste focados onde os erros mais provavelmente ocorrem (erros de $<$ vs $\leq$, off-by-one, etc.).

Vantagem: alta taxa de detecção de defeitos com poucos testes.

CLASSE	Por quê?
17	Logo abaixo do limite inferior
18	Exatamente no limite inferior
19	Logo acima do limite inferior
64	Logo abaixo do limite superior
65	Exatamente no limite superior
66	Logo acima do limite superior

Refere-se à abordagem de derivar casos de teste de forma metódica e reproduzível, seguindo critérios bem definidos em oposição a testes ad-hoc ou baseados em intuição (como Error Guessing).

As técnicas anteriores (Particionamento e Valor Limite) são exemplos de abordagens sistemáticas. A ideia geral é:

- Seguir um processo definido para gerar os testes
- Ter critérios de cobertura mensuráveis (ex: "todas as classes de equivalência cobertas")
- Ser reproduzível, ou seja, outro testador seguindo o mesmo método chega aos mesmos casos de teste
- Ter justificativa para cada caso de teste (não é um "chute")

Técnica sistemática para derivar casos de teste a partir de combinações de condições de entrada (causas) e seus resultados (efeitos).

Como funciona:

- **Identifique as causas:** condições de entrada ou estímulos (ex: "usuário autenticado", "saldo > 0")
- **Identifique os efeitos:** saídas ou ações do sistema (ex: "transação aprovada", "mensagem de erro")
- **Construa o grafo:** conecte causas aos efeitos usando operadores lógicos (AND, OR, NOT)
- **Adicione restrições:** nem todas as combinações de causas são possíveis (ex: exclusividade mútua)
- **Gere uma tabela de decisão:** cada coluna é um caso de teste derivado do grafo

Vantagem: força você a pensar em combinações que testes *ad-hoc* ignorariam. É especialmente útil quando o comportamento depende de múltiplas condições simultâneas.

Exemplo:

Causa 1: idade ≥ 18

Causa 2: tem CNH

Efeito: pode alugar carro (C1 AND C2)

O grafo gera 4 casos: (V,V), (V,F), (F,V), (F,F)

Técnica baseada na experiência e intuição do testador. Não segue um processo formal, o testador "adivinha" onde os bugs provavelmente estão.

O testador usa seu conhecimento sobre:

- Erros comuns de programação
- Histórico de bugs do sistema
- Pontos frágeis típicos

... para criar casos de teste direcionados.

Exemplos clássicos de "chutes" que costumam encontrar bugs:

- Divisão por zero
- Strings vazias ou null
- Listas vazias ou com um único elemento
- Valores nos limites (0, -1, MAX_INT)
- Datas como 29/02, 31/04, virada de ano
- Campos com caracteres especiais (' , " , <script>)
- Arquivo vazio, arquivo muito grande
- Conexão de rede perdida no meio de operação

Vantagem: rápida, complementa técnicas formais, encontra bugs que métodos sistemáticos podem perder.

Desvantagem: depende totalmente da experiência do testador, não é reproduzível nem garante cobertura.

Uma loja online oferece descontos que variam conforme o valor da compra e o tipo de cliente. Compras abaixo de R\$100 não recebem desconto, independentemente do cliente. Para compras entre R\$100 e R\$499, clientes regulares recebem 10% de desconto, clientes premium recebem 15% e funcionários recebem 20%. Já para compras a partir de R\$500, os descontos aumentam para 15%, 25% e 30%, respectivamente. Valores negativos de compra e tipos de cliente não cadastrados são considerados inválidos pelo sistema.

1. PARTICIONAMENTO DE EQUIVALÊNCIA

- ✓ Objetivo: é particionar o domínio de entrada de forma que os casos de testes dos subconjuntos (partições ou subdomínios) seja uma boa representação de todo o domínio
- ✓ Divide o domínio de entrada em **classes de equivalência** que, de acordo com a especificação do programa, são tratadas da mesma maneira.
 - Isto é, o valor dentro de uma classe é equivalente a todos os outros valores dentro da mesma classe
 - Logo, se eu testar um valor de uma classe, eu estarei cobrindo os outros valores dessa mesma classe
- ✓ Assumption: Se um elemento do subconjunto detectar um defeito, qualquer outro também detecta

- ✓ Pode-se observar a especificação procurando termos como:
 - “*intervalo*” e “*conjunto*” ou
 - Palavras similares que indiquem que os dados são processados da mesma forma
- ✓ Representa um conjunto de estados válidos ou inválidos para as condições de entrada

- ✓ Diretrizes para a definição das classes:
 - Se a condição de entrada especifica um **intervalo de valores**, são definidas uma classe válida e duas inválidas
 - Se a condição de entrada especifica uma **quantidade de valores**, são definidas uma classe válida e duas inválidas
 - Se a condição de entrada especifica um **conjunto de valores determinados manipulados de forma diferente**, é definida uma classe válida para cada um desses valores e uma classe inválida com outro valor qualquer
 - Se a condição de **entrada especifica uma situação do tipo “deve ser de tal forma”**, são definidas uma classe válida e uma inválida
- ✓ Definidas as classes, deve-se definir casos de teste que cubram o maior número de classes

- ✓ 1º passo: definir classes de equivalência
- ✓ 2º passo: definir casos de teste de forma a ter o menor número de casos de teste possível
- ✓ Considerando a especificação dada anteriormente, têm-se **duas entradas**:
 - **V** – valor da compra
 - **C** – tipo de cliente

- ✓ O valor da compra (**V**) forma três classes válidas e uma inválida.
 - ✓ (**V1**) – R\$0 – R\$99,99
 - ✓ (**V2**) – R\$100 – R\$499,99
 - ✓ (**V3**) – R\$500+
 - ✓ (**I1**) – < R\$0 (negativo)
- ✓ O tipo de cliente (**C**) também forma três classes válidas e uma inválida.
 - ✓ (**C1**) – Regular
 - ✓ (**C2**) – Premium
 - ✓ (**C3**) – Funcionário
 - ✓ (**I1**) – Desconhecido/nulo

- ✓ Como temos duas entradas, precisamos considerar as combinações. Para classes válidas, fazemos o produto cartesiano:

CASO	VALOR (CLASSE)	CLIENTE (CLASSE)	DESCONTO ESPERADO
1	R\$50 (V1)	Regular (T1)	0%
2	R\$50 (V1)	Premium (T2)	0%
3	R\$50 (V1)	Funcionário (T3)	0%
4	R\$250 (V2)	Regular (T1)	10%
5	R\$250 (V2)	Premium (T2)	15%
6	R\$250 (V2)	Funcionário (T3)	20%
7	R\$800 (V3)	Regular (T1)	15%
8	R\$800 (V3)	Premium (T2)	25%
9	R\$800 (V3)	Funcionário (T3)	30%

- ✓ Para classes inválidas, testamos uma de cada vez (mantendo a outra entrada válida):

CASO	VALOR (CLASSE)	CLIENTE (CLASSE)	DESCONTO ESPERADO
10	-R\$50 (I1)	Regular (T1)	Erro: valor inválido
11	R\$200 (V2)	"ouro" (I2)	Erro: tipo inválido

✓ **Vantagem:** resulta em casos de teste reduzidos

- OK quando as entradas podem ser facilmente identificadas na especificação do problema

✓ **Desvantagem:**

- Quando a entrada é simples, mas o processamento é complexo
- Grupos de dados de mesma partição podem não ser processados de forma idêntica

2. ANÁLISE DO VALOR LIMITE

- ✓ Partindo das mesmas classes de equivalência, agora focamos nos valores exatamente nas fronteiras.

Fronteiras do Valor da Compra: Temos 3 fronteiras: R\$0, R\$100 e R\$500.

CASO	VALOR	FRONTEIRA	DESCONTO ESPERADO
1	-R\$0,01	Logo abaixo de R\$0	Inválido
2	R\$0,00	Exatamente em R\$0	Sem desconto
3	R\$0,01	Logo acima de R\$0	Sem desconto
4	R\$99,99	Logo abaixo de R\$100	Sem desconto
5	R\$100,00	Exatamente em R\$100	10% (regular)*
6	R\$100,01	Logo acima de R\$100	10% (regular)*
7	R\$499,99	Logo abaixo de R\$500	10% (regular)*
8	R\$500,00	Exatamente em R\$500	15% (regular)*
9	R\$500,01	Logo acima de R\$500	15% (regular)*

* É necessário repetir para todos os tipos de funcionários.

3. TESTE FUNCIONAL SISTEMÁTICO

- ✓ Combina os critérios de *Particionamento de Equivalência & Análise do Valor Limite*
- ✓ Este critério requer **ao menos dois casos de teste** de cada partição
 - Minimiza que defeitos coincidentes mascarem falhas
- ✓ Diretrizes, tanto para valores de entrada quanto de saída:
 - Para **valores discretos**: testar todos os valores
 - Para **intervalos de valores**: testar os extremos e um valor no interior
 - Se valor tem 16 bits, os valores -32768 e +32767 devem ser selecionados
 - Números reais: deve-se definir uma margem de erro, selecionar valores bem pequenos e zero
 - Texto: deve explorar comprimentos variáveis, alfabéticos, alfanuméricos, caracteres especiais
- ✓ Valores imediatamente **fora do limite** para verificar se o software os rejeita

- ✓ Por ser baseado nos critérios Particionamento de Equivalência e Análise do Valor Limite, o critério Funcional Sistemático apresenta os mesmos problemas que estes
- ✓ Fornece diretrizes para facilitar a geração de casos de testes e enfatiza mais fortemente a seleção de mais de um caso de teste por partição e/ou limite
 - Maior chance de revelar os defeitos
 - Maior probabilidade de obter uma melhor cobertura de código

- ✓ Uma vantagem do teste funcional é que ele se baseia somente na especificação e não depende da forma que o software foi implementado:
 - Procedimental, orientado a objetos, ou componentes
- ✓ ENTRETANTO:
 - E se a especificação está incompleta?
 - E se os elementos de uma mesma classe de equivalência NÃO se comportam da mesma maneira?
 - Também, esse tipo de teste não assegura que partes do código foram cobertas pelos casos de teste

- ✓ Façam o particionamento de classes de equivalência do problema dos triângulos visto na aula passada.
- ✓ Proponham testes baseados no critério Funcional Sistemático (particionamento de classes + análise do valor limite).
- ✓ Implementem testes no ambiente Unity apresentado na aula passada.
- ✓ **A função fornecida possui algum erro segundo os testes de vocês?**

✓ Entradas Inválidas

- Lado não-positivo: pelo menos um dos lados ≤ 0
- Violação da Desigualdade Triangular:
 - $a + b \leq c$
 - $a + c \leq b$
 - $b + c \leq a$
- Risco de overflow: se a soma de dois lados estoura o tipo antes de comparar.

✓ Entradas Válidas

- Que produz saída “Equilátero”
 - lados positivos onde $a == b == c$
- Que produz saída “Isósceles”
 - lados positivos, desigualdade satisfeita, e exatamente dois lados iguais
- Que produz saída “Escaleno”
 - lados positivos, desigualdade satisfeita, onde $a \neq b \neq c \neq a$